3 محمد عبد الإله محمد الدلاعين

المقارنة · Round Robin · Context switch

📌 شو إلك بالمشروع

المادة نظم تشغيل: ابدأ دايماً بشرح الفكرة (المربعات البنفسجية 🧠) وبعدين ورجي كيف طبّقناها بالكود.

- **Phase 2 · فيديو 19 نوفمبر:** خوارزمية Round Robin
- **Phase 3 · 17 ديسمبر:** تكلفة الـ Context switch ومقارنة الخوارزميات

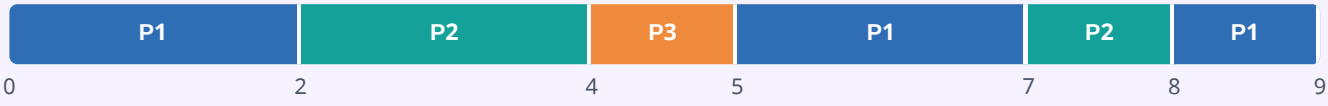
الكود كله بملف **CPU_SchedLab.py**. افتحه بـ VS Code واكبس **Ctrl + G** واكتب رقم السطر اللي بالصورة.

Phase 2 · فيديو 19 نوفمبر خوارزمية Round Robin

🧠 الفكرة بنظام التشغيل · خوارزمية Round Robin والـ Time sharing

Round Robin معمولة لأنظمة الـ **time sharing**، اللي فيها كذا مستخدم أو برنامج بدهم يحسّوا إنهم شغّالين بنفس الوقت. الـ ready queue بيكون **طابور دائري FIFO**، وكل process بتاخد شريحة وقت ثابتة اسمها **time quantum**. إذا خلصت قبلها بتطلع، وإذا ما خلصت بيصير **timer interrupt**، والنظام بياخد منها المعالج (preemptive) وبيرجّعها لآخر الطابور. اختيار الـ quantum مهم كتير: إذا كبير بتصير زي FCFS، وإذا صغير كتير الوقت بيروح على الـ context switch. القاعدة بالكتاب إنه **80%** **من الـ CPU bursts** لازم يكونوا أقصر من الـ quantum. وميزتها الأكبر إنه الـ **response time** قليل.

Process	Arrival	Burst	Completion	Turnaround	Waiting	Response
P1	0	5	9	9	4	0
P2	1	3	8	7	4	1
P3	2	1	5	3	2	2



🧮 **1** = 3 ÷ (0 + 1 + 2) response · **3.33** = 3 ÷ (2 + 4 + 4) waiting ← q = 2، أحسن من FCFS (3.33)

خوارزمية Round Robin (F6)

CPU_SchedLab.py lines 168–197

```python
def round_robin_schedule(processes, options):
    """F6 Round Robin: each process runs at most `quantum`, then goes to the back."""
    cpu = Dispatcher(options.switch_cost)
    arrivals = sorted(processes, key=lambda p: p.arrival)
    remaining = {p.pid: p.burst for p in processes}
    queue = deque()                                  # the ready queue (FIFO)
    time = 0

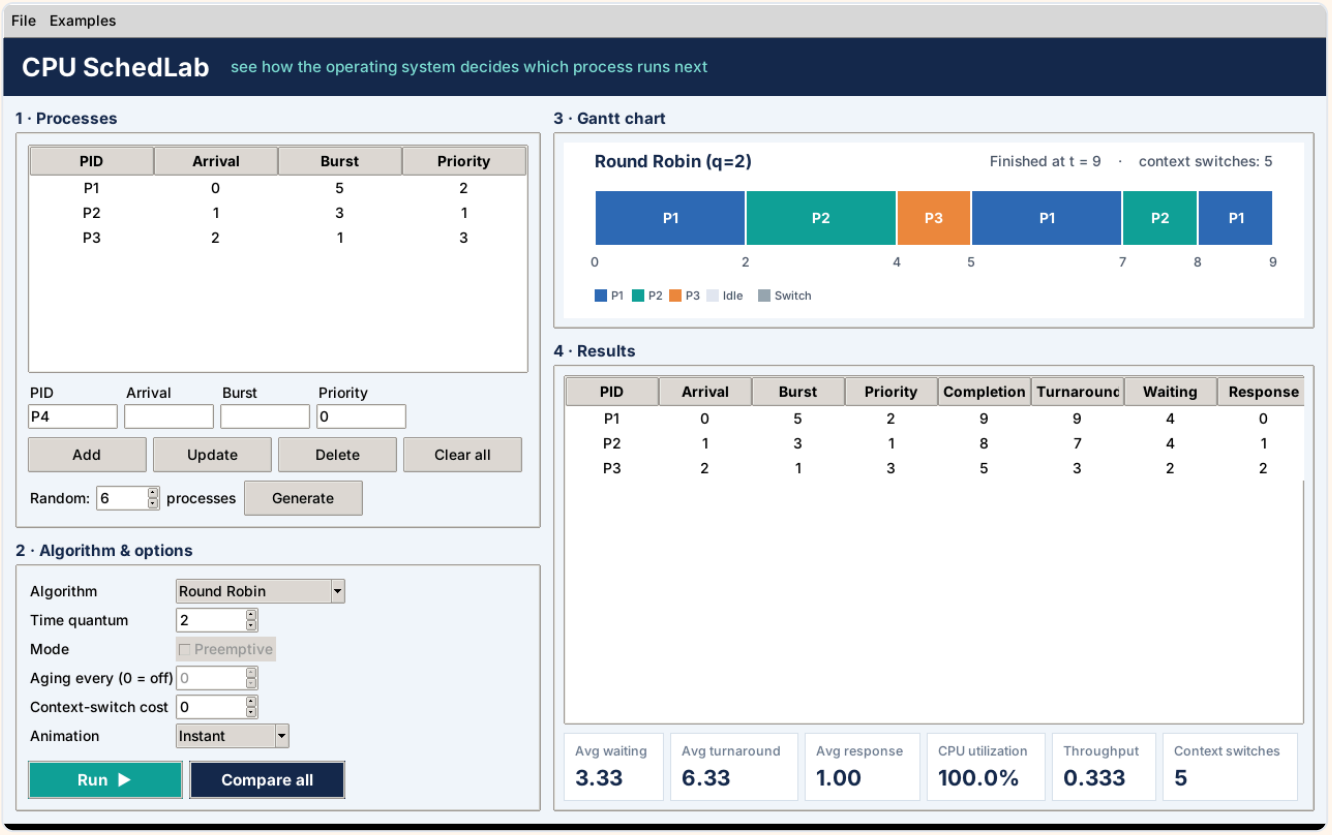
    def admit(up_to):                                # arrived processes join the queue
        while arrivals and arrivals[0].arrival <= up_to:
            queue.append(arrivals.pop(0))

    admit(time)
    while queue or arrivals:
        if not queue:                                # queue empty -> idle
            cpu.idle(time, arrivals[0].arrival)
            time = arrivals[0].arrival
            admit(time)
            continue
        p = queue.popleft()                          # take the FRONT process
        time = cpu.dispatch(p.pid, time)
        admit(time)
        run_for = min(options.quantum, remaining[p.pid])    # one quantum (or less)
        cpu.run(p.pid, time, time + run_for)
        time += run_for
        remaining[p.pid] -= run_for
        admit(time)                                  # new arrivals enter first
        if remaining[p.pid] > 0:
            queue.append(p)                          # not finished -> back of queue
    return cpu.result(f"Round Robin (q={options.quantum})", processes)
```

🎤 شو بتحكي للدكتورة

خوارزمية **Round Robin** بتمثّل فكرة الـ **time sharing**. الـ ready queue هون طابور حقيقي **deque** بيمشي FIFO. الدالة **admit** بتدخّل عالطابور كل process وصل وقتها. بكل دورة باخد الـ process اللي **بأول** الطابور وبشغّلها مدة الـ **quantum**، أو أقل إذا ضايلها أقل. إذا ما خلصت بترجع **لآخر** الطابور، وبنتبه إنه الـ processes اللي وصلت هسا بتفوت قبلها. جرّبناها على مثال الكتاب والـ quantum بـ 4، فطلع معدّل الانتظار **5.67**، نفس جواب الكتاب. إذا كان الـ quantum كبير كتير بتصير زي FCFS، وإذا كان صغير كتير بيروح الوقت بالـ context switch.

🖥️ شو بتعرض عالشاشة



بختار Round Robin والـ quantum بـ 2، وبكبس **Run**: بنشوف كيف الـ processes بتتناوب على المعالج: P1، P2، P3، P1، P2، P1. وبعدين من قائمة Examples بختار مثال الكتاب، والـ quantum بـ 4، بيطلع المعدّل **5.67**.

💬 إذا سألتك الدكتورة

❓ شو بيصير إذا الـ quantum كبير كتير؟

بتصير زي FCFS.

❓ وإذا صغير كتير؟

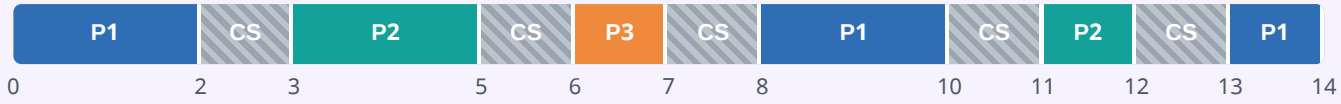
كتير context switches، والوقت بيروح عالتبديل. القاعدة إنه 80% من الـ bursts يكونوا أقصر من الـ quantum.

❓ ليش Round Robin منيحة؟

لأنه الـ response time تبعها قليل، فبتناسب الأنظمة التفاعلية.

🧠 الفكرة بنظام التشغيل · الـ Context switch والـ Dispatch latency

الـ **context switch** هو لما المعالج ينتقل من process لـ process: النظام بيحفظ الـ **state** تبعت القديمة (الـ registers والـ program counter) بالـ **PCB** تبعها، وبيحمّل الـ state تبعت الجديدة من الـ PCB تبعها. هاد الوقت **overhead خالص**: المعالج ما بيعمل فيه ولا اشي مفيد لأي process. والوقت اللي بياخده الـ dispatcher ليوقّف process ويشغّل غيرها اسمه **dispatch latency**. بالمثال تحت عملنا Round Robin بـ q = 2 وتكلفة تبديل = 1: صار عنا 5 تبديلات، والوقت الكلي صار 14 بدل 9، و CPU utilization نزل من 100% لـ **64%**. عشان هيك الـ quantum لازم يكون أكبر بكتير من وقت الـ context switch.



🧮 CPU utilization = وقت الشغل المفيد ÷ الوقت الكلي = 9 ÷ 14 = **64%**

الـ Dispatcher والـ Context switch (F11)

CPU_SchedLab.py lines 200–231

```python
class Dispatcher:
    """F11 Builds the timeline for EVERY algorithm and handles context switches."""

    def __init__(self, switch_cost=0):
        self.switch_cost = switch_cost
        self.timeline = []
        self.switches = 0
        self.last_pid = None

    def dispatch(self, pid, now):
        """Give the CPU to `pid`. A different process than before = context switch."""
        if self.last_pid is not None and pid != self.last_pid:
            self.switches += 1                              # save old PCB, load new PCB
            if self.switch_cost > 0:
                self.add(SWITCH, now, now + self.switch_cost)
                now += self.switch_cost                     # time lost (dispatcher latency)
        self.last_pid = pid
        return now                                          # when the process really starts

    def run(self, pid, start, end):
        self.add(pid, start, end)

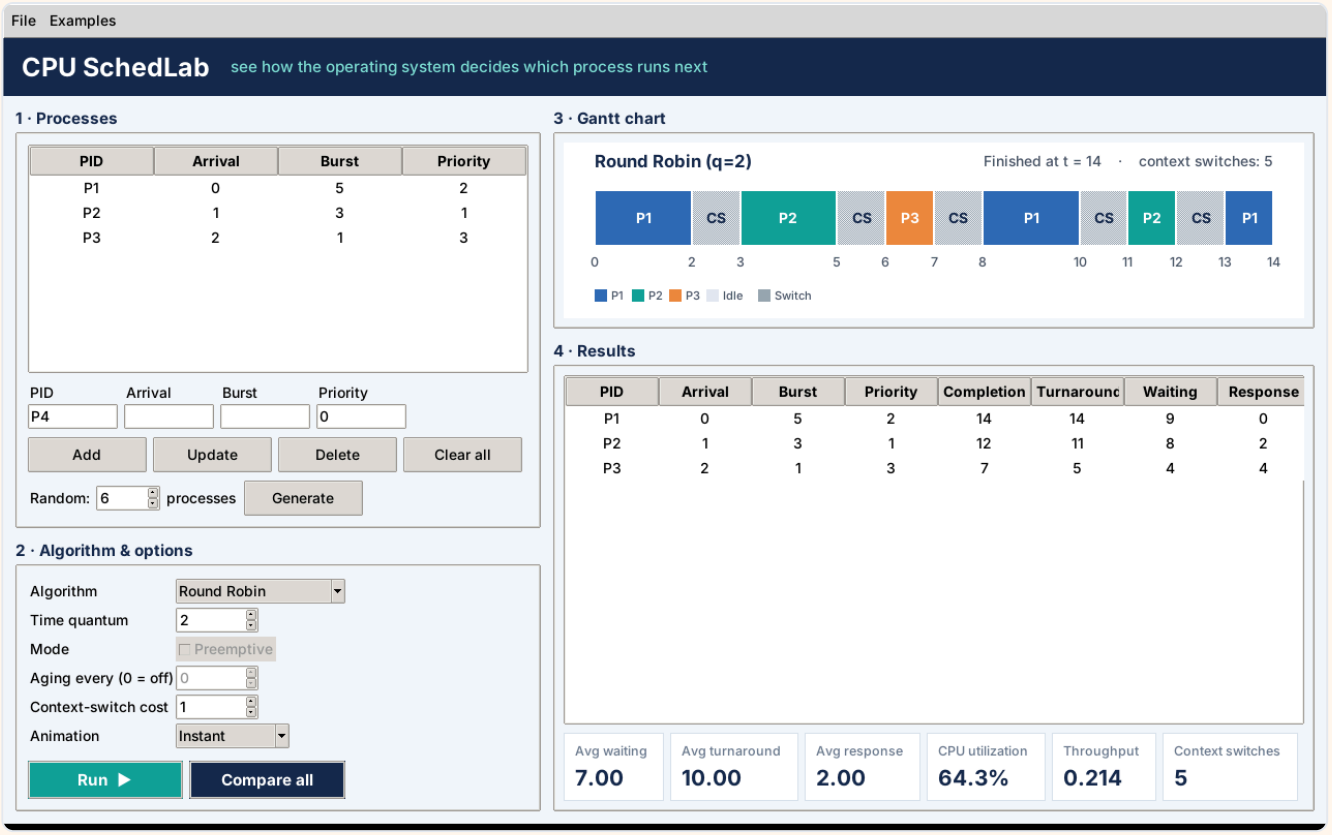
    def idle(self, start, end):
        if end > start:
            self.add(IDLE, start, end)

    def add(self, pid, start, end):
        last = self.timeline[-1] if self.timeline else None
        if last and last.pid == pid and last.end == start:  # same process continues
            last.end = end                                  # -> make the bar longer
        else:
            self.timeline.append(Segment(pid, start, end))
```

🎤 شو بتحكي للدكتورة

الكلاس **Dispatcher** بتستخدمه الخمس خوارزميات كلهم عشان يبنوا الخط الزمني. أهم دالة فيه **dispatch**: لما ننتقل من process لـ process غيرها بيصير **context switch**، يعني النظام بيحفظ حالة القديمة بالـ PCB تبعها وبيحمّل حالة الجديدة. أنا بعدّ هالمرات، وإذا المستخدم حط تكلفة للتبديل بضيف مستطيل رمادي اسمه **CS** بالرسم، وبأخّر بداية الـ process، وهاد اللي بنسمّيه **dispatcher latency**. أما الدالة **add**، فإذا نفس الـ process كمّلت شغل بتطوّل المستطيل، بدل ما ترسم مستطيلات صغيرة كتير.

🖥️ شو بتعرض عالشاشة



بحط الـ quantum بـ 2 وتكلفة التبديل بـ 1 وبكبس **Run**: بتطلع مستطيلات **CS** الرمادية، صار عنا 5 تبديلات، والوقت الكلي 14 بدل 9، ونسبة استخدام المعالج نزلت لـ **64%**. يعني الـ context switch وقت ضايع.

🧠 الفكرة بنظام التشغيل · تقييم الخوارزميات (Algorithm evaluation)

الكتاب بيحكي إنه في كذا طريقة نقيّم فيها خوارزميات الجدولة: **deterministic modeling** يعني بناخد workload ثابت وبنحسب أداء كل خوارزمية عليه، و**queueing models** يعني معادلات رياضية، و**simulation** يعني برنامج بيحاكي النظام، و**implementation** يعني نجرّبها بنظام حقيقي. مشروعنا بيعمل **deterministic modeling و simulation**: نفس الـ processes بالزبط بتمرق على الخمس خوارزميات، وبنقارن المعايير. والنتيجة دايماً إنه ما في خوارزمية أحسن بكل اشي: SRTF أحسن بالانتظار، و RR أحسن بالـ response، و FCFS أبسط وأقل تبديلات.

مقارنة كل الخوارزميات (F10)

CPU_SchedLab.py — lines 237–243

```python
def compare_all(processes, options):
    """F10 Run EVERY algorithm on the SAME processes and collect the averages."""
    summaries = []
    for name in ALGORITHMS:
        _, summary = compute_metrics(run(name, processes, options))
        summaries.append(summary)
    return summaries
```

🎤 شو بتحكي للدكتورة

الدالة **compare_all** هي المقارنة: بتشغّل الخمس خوارزميات على **نفس الـ processes بالزبط**، وبتجمع المعدّلات لكل وحدة، وهاد اللي بيسمّيه الكتاب **deterministic modeling**. وبعدين الرسومات بتنعمل بـ **Matplotlib**: أربع رسومات أعمدة لمعدّل الانتظار والـ turnaround والـ response وعدد التبديلات، والأحسن بكل رسمة بيكون حواليه إطار أسود.

🖥️ شو بتعرض عالشاشة



بكبس **Compare all**، بتطلع نافذة المقارنة، والنجمة ★ على الخوارزمية اللي إلها أقل معدّل انتظار، وهي SRTF بمثالنا. وبنلاحظ إنه Round Robin مش الأحسن بالانتظار، بس منيحة بالـ **response time**.